

# A1 — Data points, features and labels from one photograph

MLR1ILV — Machine Learning 1 · IMC Krems, WS26/27

Starts in Session 1 (Wed 30.09) + Session 2 (Thu 01.10) · **due Wed 07.10.2026, 23:59** · 100 points

## WHAT THIS IS ABOUT

A spreadsheet arrives with its data points already decided: one per row. A photograph does not. Before you can train anything, **you** have to say what one data point is. That choice fixes how many data points you have available to train a ML model. It also fixes what a feature is, and what a label can mean. This assignment requires you to do this choice, on **7.3 km of the Wachau seen from the air**: Weissenkirchen, Rossatz, Dürnstein and the Loiben slopes in one photograph.

The labels come from INVEKOS, the Austrian register of agricultural parcels, in which every farmer declares the fields they farm in order to receive subsidies (AMA, open data, CC BY 3.0 AT). The parcel outlines were drawn onto the photograph's pixel grid; the result is the mask `wachau_labels.png`: 0 no parcel, 1 vineyard, 2 other crop.



The photograph:  $9216 \times 4608$  pixels at 0.79 m per pixel, 7.3 km of the Wachau with Weissenkirchen, Rossatz and Dürnstein. The vine rows show as stripes. `get_wachau_photo.py` (attached) downloads it once.

(Datenquelle: basemap.at, CC BY 4.0)



The parcel mask `wachau_labels.png`, built by the same script on the same pixel grid: white = 0, no parcel; **green** = 1, vineyard; **red** = 2, other crop. (INVEKOS Schläge 2025, AMA, CC BY 3.0 AT)

### WHAT TO DO

- 1) **Get the data.** Run `uv run get_wachau_photo.py` once. It writes two files next to yours: the photograph `wachau_ortho.jpg` (from `basemap.at`) and the parcel mask `wachau_labels.png` (built from the INVEKOS web service). Then run `uv run A1.py`: the starter code loads both into the arrays `photo[row, column, channel]` and `labels[row, column]` and prints their shapes and the label histogram. Open both image files and look at them before you write any code.
- 2) **In `A1.py`: three candidate data points, all three computed.** (a) One *pixel*: its feature vector is its three colour values, so  $m = 42,467,328$  and  $d = 3$ . (b) One *patch* of  $P \times P$  pixels: start with  $P = 32$  (25 m on the ground), cut the grid into patches and compute a few features per patch, for example the mean and the standard deviation of each colour channel. (c) The *whole image*:  $m = 1$ . Print  $m$  and  $d$  for each of the three.
- 3) **In `A1.py`: the label of each data point.** A pixel's label is its value in `labels`. A patch's label is *vineyard* if the share of its pixels with value 1 exceeds a threshold. Compute that share for every patch, and print the share of patches labelled vineyard for the thresholds  $>25\%$ ,  $>50\%$  and  $>75\%$ . Everything with value 0 (no parcel) counts as *not vineyard*.
- 4) **In `A1.py`: the baseline.** For the pixel data points and for the patch data points at each threshold, print the vineyard share and the accuracy of the constant prediction of the majority class. That accuracy is the baseline every later result is compared with.

- 5) In `A1_whytrail.txt`, **section 4: your decisions**. Write down (i) your data point: pixel or patch, and  $P$ ; (ii) your label rule and the threshold you chose, with one sentence why; (iii) one sentence on what counting value 0 as *not vineyard* does to the task (forest, roads and undeclared fields all become *not vineyard*); (iv) one sentence on why the whole image is not a usable data point; (v)  $m$ ,  $d$  and the baseline that follow from your choices, copied from the output of `A1.py`.
- 6) In `A1_whytrail.txt`, **sections 1 to 3: how the code came about**. The prompts you gave the LLM, verbatim; at least one thing that was wrong in what came back; and the check that showed it. Section 5 is the hand computation from the last page of this sheet.
- 7) **Hand in** `A1.py` and `A1_whytrail.txt` on the A1 Aufgabe in MS Teams and press **Abgeben**. Two files, no ZIP, no other file names.

#### WHAT YOU HAND IN

Two files, **no ZIP**, attached to this Aufgabe in **MS Teams** → **Aufgaben** → **A1**, **Abgeben** pressed by **Wed 07.10.2026, 23:59**:

- `A1.py` — your program. Run it with `uv run A1.py`.
- `A1_whytrail.txt` — the why-trail, your written log of what you asked for, what came back and what you checked, in five sections.

Both files are attached to this Aufgabe as templates: edit them, do not rename them. The templates carry everything else you need — how to get Python running (in `A1.py`) and how the assignment is scored and handed in (in `A1_whytrail.txt`).

Already attached to this Aufgabe, ready to download:

- `get_wachau_photo.py`

#### HOW IT IS SCORED

| Part                                              | Points    | What it checks                 |
|---------------------------------------------------|-----------|--------------------------------|
| Artifact: it runs (15) + a defensible result (10) | 25        | LLM help expected              |
| <b>Why-trail</b> , sections 1–4                   | <b>45</b> | <b>your</b> verification steps |
| Hand computation, section 5                       | 30        | you understand it yourself     |

Why-trail 45 = a real flaw found (20) + the result justified against a stated baseline (15) + readable and plain about what is still uncertain (10). **Copied code with a why-trail that contains no checks of your own caps at 25**, and cannot be explained at the five-minute spot-check. Using an LLM is **expected**. Being misled by it is not.

## SECTION 5 — THE HAND COMPUTATION (30 POINTS)

Work this **on paper**. No code, no LLM. Write the answer *and* the working into section 5 of your `A1_whytrail.txt`. Part 1 of the written exam asks you to do exactly this, so these six questions are the six practice runs.

**The label is a decision, and the baseline comes before the model**

Eight patches from the tile, each one data point. For each, the share of its area covered by a parcel, and what that parcel is:

| patch        | 1  | 2    | 3    | 4  | 5    | 6  | 7     | 8    |
|--------------|----|------|------|----|------|----|-------|------|
| parcel cover | 0% | 90%  | 100% | 0% | 60%  | 0% | 30%   | 95%  |
| parcel type  | —  | vine | vine | —  | vine | —  | maize | vine |

- 1) Label all eight data points using “vineyard if **more than half** the patch is vineyard”.
- 2) What is the vineyard share?
- 3) What accuracy does the constant prediction “not vineyard” have on these eight data points?
- 4) One sentence: if you had instead kept **only the patches a parcel covers at all**, what would the vineyard share have become, and why does that number mislead?

## DICTIONARY ENTRIES FOR THIS ASSIGNMENT

The five entries to read alongside it, from the **Dictionary of Applied Machine Learning** ([dictionaryofml.org](https://dictionaryofml.org)). Every key and entry name below is a link to that term’s page.

| key                       | entry             | what it settles                                                           |
|---------------------------|-------------------|---------------------------------------------------------------------------|
| <a href="#">datapoint</a> | <b>data point</b> | the unit you predict a label for; here one patch, one pixel, or the image |
| <a href="#">feature</a>   | <b>feature</b>    | a measurable property of a data point, as a number                        |
| <a href="#">label</a>     | <b>label</b>      | what you wish you knew; here: vineyard yes/no                             |
| <a href="#">loss</a>      | <b>loss</b>       | how wrong one prediction is on one data point, and what that costs whom   |
| <a href="#">baseline</a>  | <b>baseline</b>   | the accuracy of the constant prediction of the majority class             |

Deadline clashing with work or family? Message me in Teams *before* it passes.